Implementing Control Structures:

An Employee Shift Scheduler in Python and Go

Xhon Pelushi

University of the Cumberlands

MSCS-632-M30: Advanced Programming Languages

Dr. Jay Thom

June 19, 2026

Abstract

This paper presents an employee shift-scheduling application implemented independently in Python and Go to demonstrate control structures—conditionals, loops, and branching—across two contrasting language paradigms. The application manages a seven-day operation with three daily shifts (morning, afternoon, and evening), enforcing that no employee works more than one shift per day, no employee exceeds five working days per week, and every shift maintains a minimum staff of two, which is met through randomized backfilling when preferences alone are insufficient. Shift conflicts, where an employee's preferred shift is already full, are resolved by reassigning the employee to another open shift the same day or carrying the request to the next day. As an extra-credit extension, employees may rank a first and second shift preference, and the Python implementation adds a Tkinter graphical interface for both building the weekly roster and viewing the generated schedule. Running both implementations against an identical thirteen-employee sample roster produced identical results for every deterministic rule and only minor, expected differences in the randomized minimum-staffing fill, confirming that the same control-flow logic was faithfully reproduced in both languages.

Overview

The assignment scenario is a company that operates seven days a week and offers employees a morning, afternoon, or evening shift each day. The application collects each employee's shift preference for every day of the week, builds a schedule that respects company staffing rules, resolves conflicts when a preferred shift is unavailable, and prints the finished weekly schedule in a readable format. The same rules and the same sample roster were implemented in two languages chosen for contrast: Python, a dynamically typed scripting language, and Go, a statically typed, compiled language with an explicit, minimal control-flow vocabulary. Implementing identical business rules twice highlights how each language expresses conditionals, loops, and branching differently while still producing the same scheduling outcome.

Company Rules Enforced

- Each employee works at most one shift per day.
- Each employee works at most five days per week.
- Every shift, every day, ends with at least two employees; shifts that fall short after preferences are honored are filled by randomly selecting employees who have not yet reached the five-day cap.
- If an employee's preferred shift is full for a given day, the scheduler looks for another open shift the same day, then carries the request over to the next day, before finally leaving the employee off for that day.
- Bonus: employees may rank a first- and second-choice shift per day, and the scheduler honors that ranking before falling back to any open shift.

Application Design

Both implementations share the same data model and the same sample input. Each employee record stores a name, a day-by-day list of one or two ranked shift preferences, a

running count of days worked, and the resulting day-by-shift schedule. Three constants drive the scheduling rules: a maximum of five working days per week, a minimum of two employees per shift, and a maximum of four employees per shift (the upper bound that makes shift conflicts possible to demonstrate). A thirteen-employee sample roster, stored as a JSON file shared by both implementations, supplies the input data.

One design decision deserves explanation: because every employee may be available all seven days but can only work five, the scheduler must decide which two days each employee rests. Assigning those rest days greedily, by simply letting each employee work until their cap is reached, causes everyone to exhaust their five days by midweek and leaves the weekend understaffed. Instead, each employee is assigned two fixed rest days using a round-robin rotation based on their position in the roster, which spreads days off evenly across the week. The rest-day assignment is treated as a soft preference, however; the hard requirements (the five-day cap and the two-person minimum) still take priority, so an employee can be called in on a rest day as a last resort if no one else is available.

Scheduling Algorithm

The scheduler processes the week one day at a time, and each day runs in three phases that rely on nested loops, multi-level conditionals, and branching fallbacks:

Phase 1 - Ranked Preference Pass

For each employee not already assigned that day, not resting, and under the five-day cap, the scheduler walks the employee's ranked shift choices in order and assigns the first one with room remaining. This is where the bonus priority ranking takes effect: an employee's second choice is only tried if the first choice's shift is already full.

Phase 1b - Conflict Resolution

If every one of an employee's ranked choices is full, the scheduler treats it as a conflict and searches the remaining shifts that day for any open seat. If one is found, the employee is reassigned and the event is recorded in the scheduling log (for example, "Leo's preferred shift was full; moved to Evening the same day"). If no shift has room at all, the request is carried over and retried first thing the next day, before that day's normal preference pass runs; an employee who still cannot be placed simply gets an extra day off.

Phase 2 - Minimum-Staffing Fill

After preferences and conflict resolution are settled for the day, any shift still below the two-person minimum is topped up by randomly selecting an eligible employee, preferring employees who are not on a designated rest day and falling back to a resting employee only if no one else qualifies. This phase repeats in a loop until the shift reaches its minimum or no eligible employee remains, in which case a warning is logged rather than silently under-staffing the shift.

Python Implementation

The Python version is split into models.py (the Employee data class and shared constants), scheduler.py (the three-phase algorithm and a text formatter for the final schedule), cli.py (a console entry point), and gui.py (the bonus Tkinter graphical interface). The control structures map directly onto Python's idioms: for loops over days, employees, and shifts; a while loop for the minimum-staffing fill; and ordinary if/elif branching for the conflict fallbacks, with Python's truthy list checks and dictionary lookups keeping the rule logic compact.

The GUI bonus lets a user build a roster interactively—adding an employee name and a ranked shift choice for each day through a dialog, or loading the same sample roster with one click—and then generate and view the formatted weekly schedule without leaving the window. The GUI reuses the exact same scheduler.py engine as the console version, so the input and output bonus and the core scheduling logic are never duplicated.

Python 3.12 - Employee Shift Scheduler (CLI)

```
$ python3 cli.py
WEEKLY SHIFT SCHEDULE
============================================================

Monday
------
  Morning   : Bob, Isla, Mia
  Afternoon : Emma, Leo
  Evening   : Carol, Frank, Grace, Jack

Tuesday
-------
  Morning   : Bob, Grace
  Afternoon : David, Frank, Isla, Karen
  Evening   : Carol, Jack, Mia

Wednesday
---------
  Morning   : Alice, Grace, Henry, Mia
  Afternoon : David, Frank, Karen
  Evening   : Carol, Jack

Thursday
--------
  Morning   : Alice, Carol, Henry
  Afternoon : David, Emma, Karen, Leo
  Evening   : Grace, Jack

Friday
------
  Morning   : Bob, David, Grace, Henry
  Afternoon : Alice, Emma, Isla, Karen
  Evening   : Leo, Carol

Saturday
--------
  Morning   : Alice, Bob, Henry, Leo
  Afternoon : David, Frank, Isla, Karen
  Evening   : Emma, Mia

Sunday
------
  Morning   : Alice, Bob, Henry, Leo
  Afternoon : Emma, Frank, Isla
  Evening   : Jack, Mia

============================================================
```

Figure 1. Python console output for the sample thirteen-employee roster.

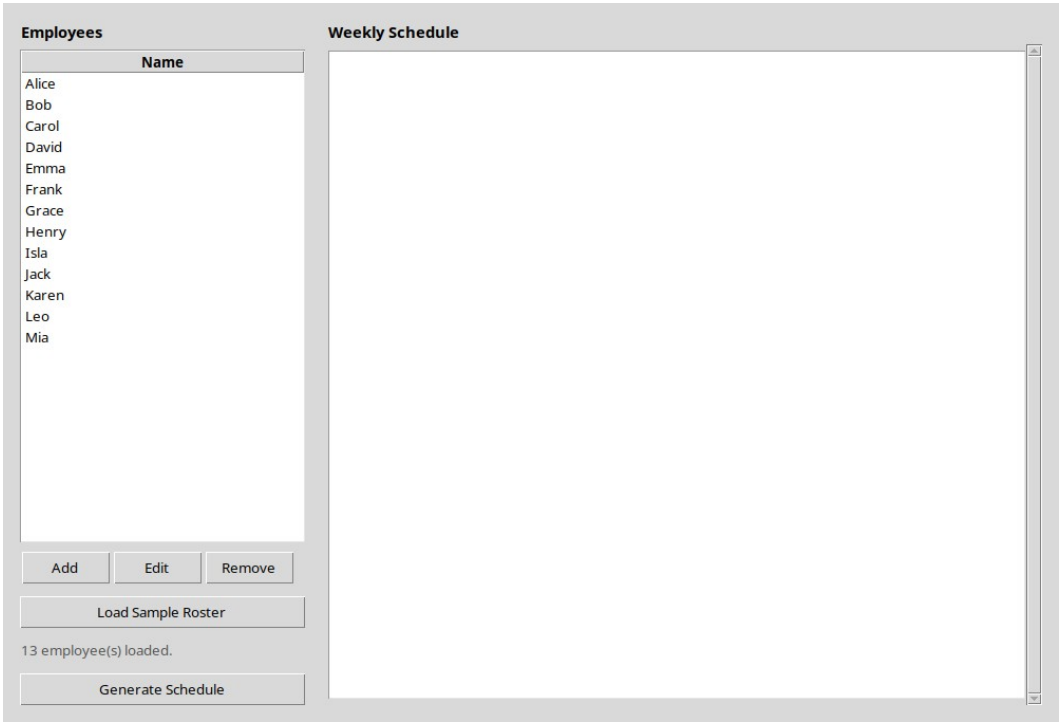

Figure 2. Python Tkinter GUI after loading the sample roster (input side).

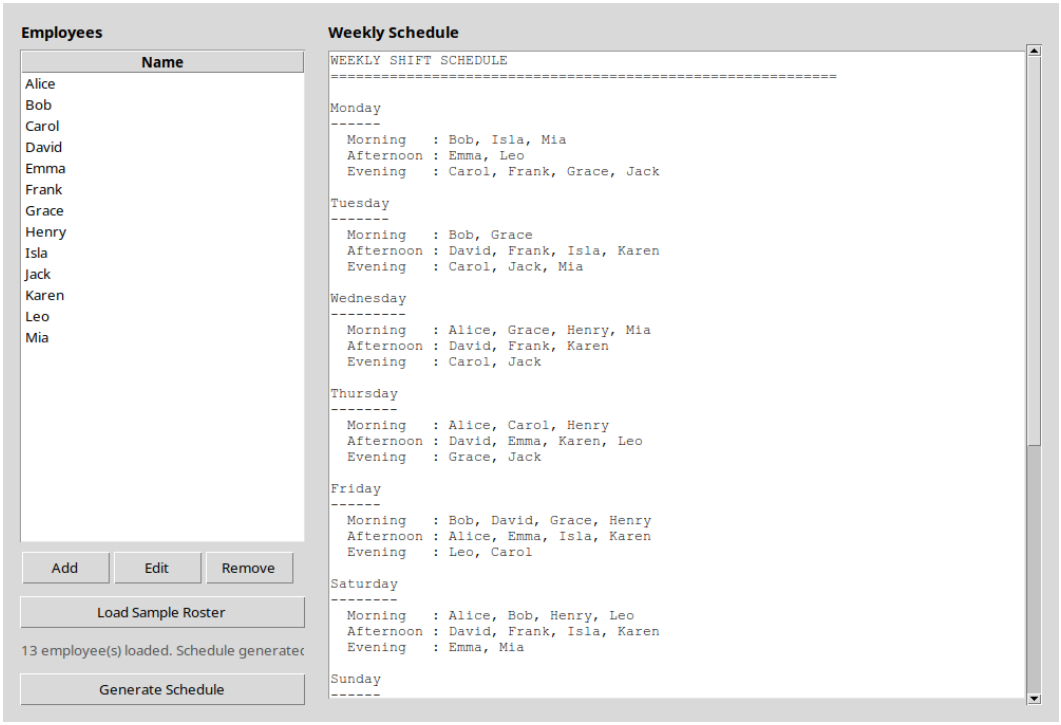

Figure 3. Python Tkinter GUI showing the generated weekly schedule (output side).

Go Implementation

The Go version mirrors the same three files conceptually—models.go, scheduler.go, and main.go—but expresses the rules in Go's idioms: explicit structs and pointer receivers in place of a dataclass, maps used as sets (map[string]bool) where Python uses a built-in set, and Go's single for keyword covering both the day/employee/shift loops and the minimum-staffing while-style loop. Go has no list-comprehension shorthand, so the same filtering Python expresses in one line (for example, selecting eligible candidates for the minimum-staffing fill) is written as an explicit loop with continue statements in eligibleCandidates. The sample roster is embedded directly into the compiled binary at build time with go:embed, a compile-time feature with no equivalent in the Python version, which instead reads the JSON file from disk at runtime.

Go 1.22 - Employee Shift Scheduler (CLI)

```
$ go run .
WEEKLY SHIFT SCHEDULE
============================================================

Monday
------
  Morning   : Bob, Isla, Mia
  Afternoon : Emma, Leo
  Evening   : Carol, Frank, Grace, Jack

Tuesday
-------
  Morning   : Bob, Grace
  Afternoon : David, Frank, Isla, Karen
  Evening   : Carol, Jack, Mia

Wednesday
---------
  Morning   : Alice, Grace, Henry, Mia
  Afternoon : David, Frank, Karen
  Evening   : Carol, Jack

Thursday
--------
  Morning   : Alice, Carol, Henry
  Afternoon : David, Emma, Karen, Leo
  Evening   : Grace, Jack

Friday
------
  Morning   : Bob, David, Grace, Henry
  Afternoon : Alice, Emma, Isla, Karen
  Evening   : Leo, Frank

Saturday
--------
  Morning   : Alice, Bob, Henry, Leo
  Afternoon : David, Frank, Isla, Karen
  Evening   : Emma, Mia

Sunday
------
  Morning   : Alice, Bob, Carol, Henry
  Afternoon : Emma, Isla, Leo
  Evening   : Jack, Mia

============================================================
```

Figure 4. Go console output for the same sample roster used by the Python version.

Cross-Language Comparison

Both implementations were run against the identical thirteen-employee sample roster. Every deterministic part of the algorithm—the ranked preference pass and the same-day conflict resolution-produced exactly the same result in both languages: every employee worked exactly five days, every shift met its two-person minimum, and the same conflict ("Leo's preferred shift was full; moved to Evening the same day" on Friday) appeared in both outputs. The one difference between the two runs was which employee was chosen to fill Friday's remaining open seat in the minimum-staffing phase—Carol in Python, Frank in Go—because Python's random module and Go's math/rand package use different pseudo-random algorithms, so an identical seed value does not produce an identical draw across languages. This is the expected outcome of comparing two independent random number generators rather than a discrepancy in the scheduling logic itself, and it was confirmed by an automated check that verified, for both languages, that no employee exceeded five working days and no shift fell below the two-person minimum.

Conclusion

Implementing the same employee shift-scheduling rules in Python and Go demonstrates how conditionals, loops, and branching carry the same logical weight across very different language designs. Python's dynamic typing and concise built-ins (sets, list comprehensions, dataclasses) produced a compact implementation, while Go's static typing and explicit control flow produced a more verbose but equally clear translation of the same three-phase algorithm. Adding the priority-ranked preference bonus required only a small change to the preference loop in both languages, and the Tkinter GUI bonus showed that the core scheduling engine could be reused unchanged behind either a console or a graphical front end.

Source Code

The complete source code for both implementations is available in the public GitHub repository: https://github.com/xhon-pelushi/MSCS-632-Advanced-Programming-Languages/tree/main/Assignment-4